

Computer Organisation & Program Execution 2019



2

Hardware/Software Interface

Uwe R. Zimmer - The Australian National University



Hardware/Software Interface

References for this chapter

[Patterson17]

David A. Patterson & John L. Hennessy

Computer Organization and Design – The Hardware/Software Interface

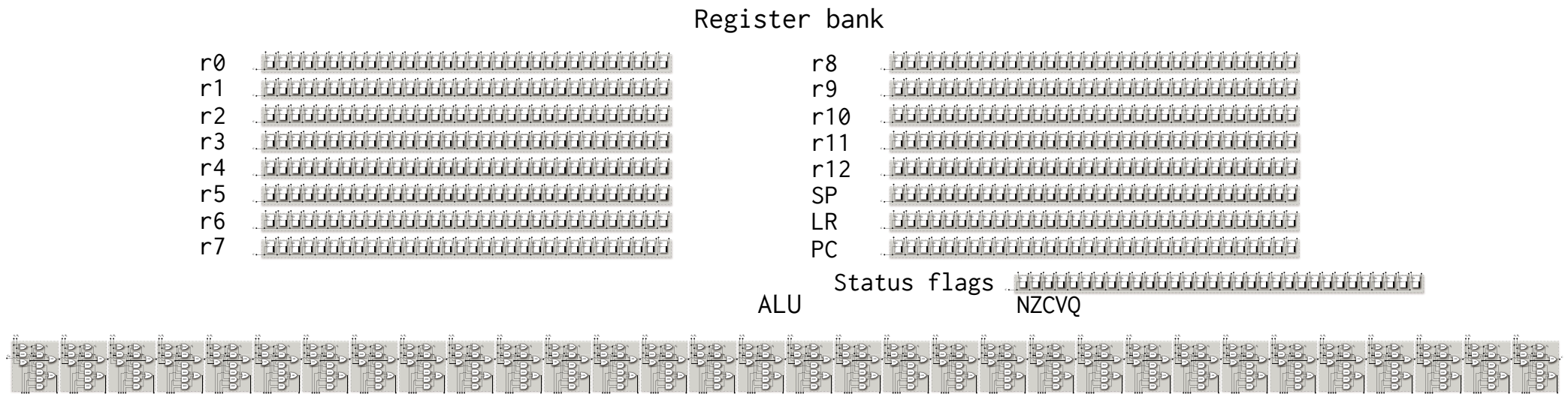
Chapter 2 “Instructions: Language of the Computer” & Chapter 3 “Arithmetic for Computers”

ARM edition, Morgan Kaufmann 2017



Hardware/Software Interface

Adding the value of two registers



The CPU will fetch the content of the memory cell which PC is pointing to.

☞ We want the CPU to execute:

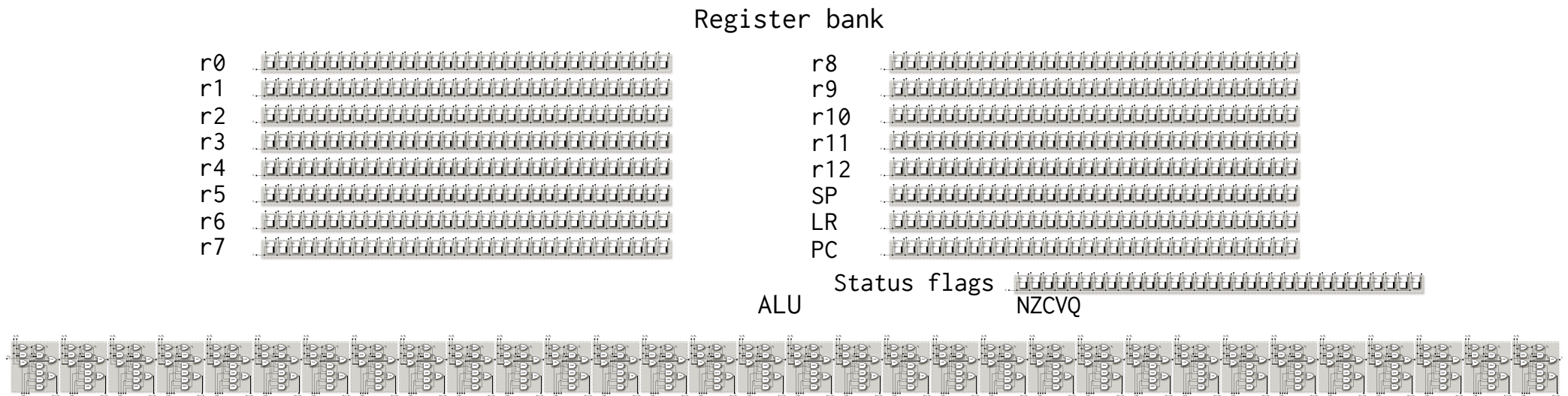
$r4 := r2 + r3$

☞ What to store in this memory cell?

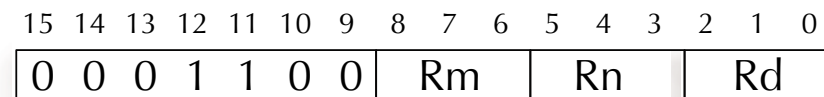


Hardware/Software Interface

Adding the value of two registers



ADDS <Rd>, <Rn>, <Rm>



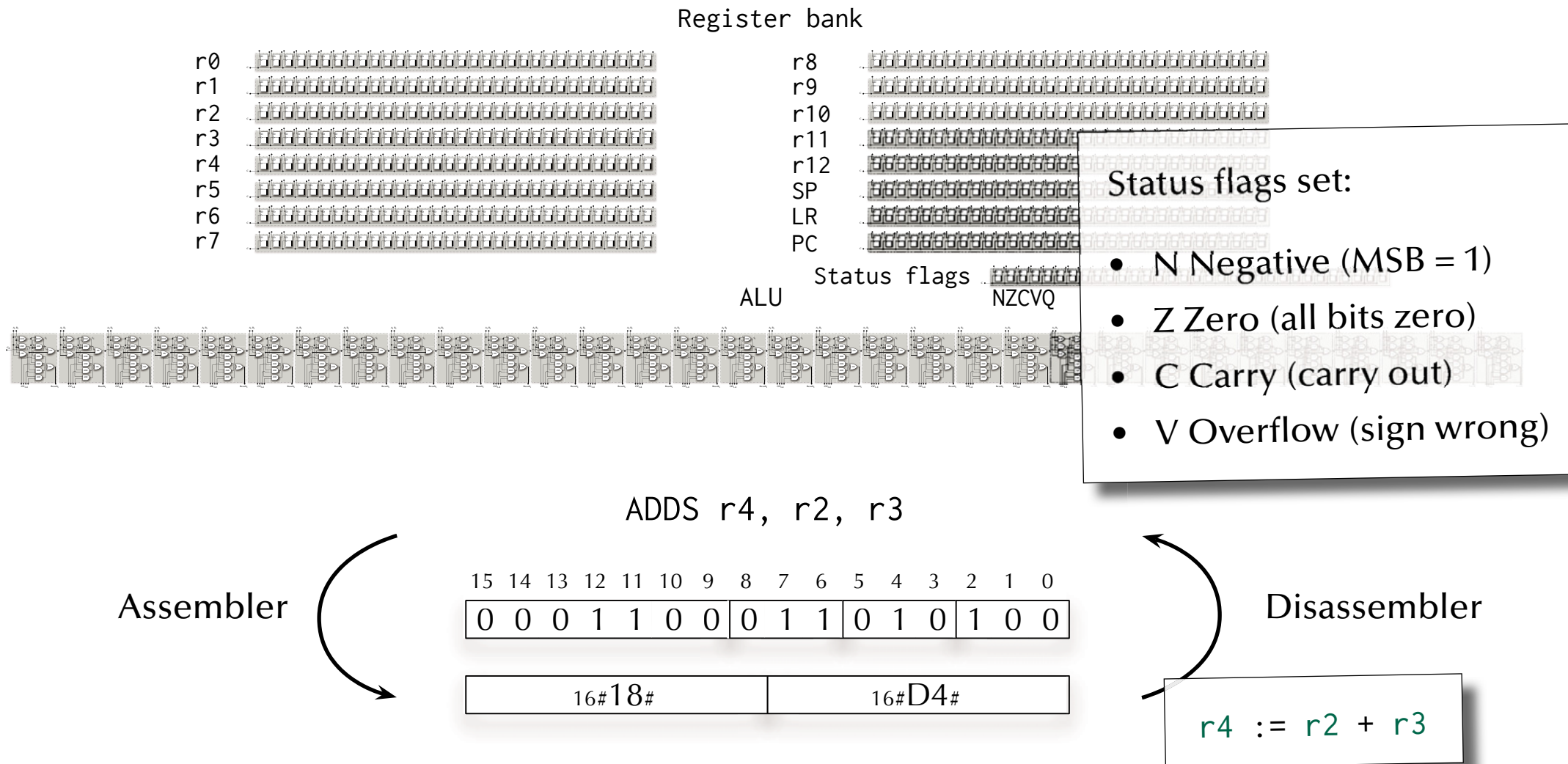
Op Code

Arguments



Hardware/Software Interface

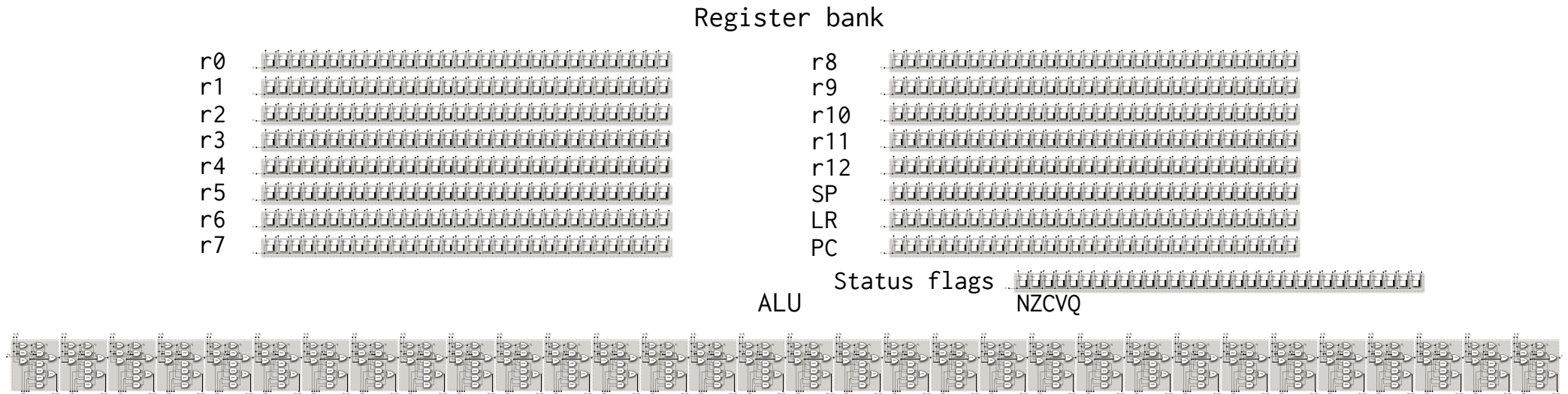
Adding the value of two registers



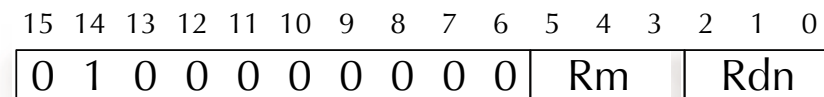


Hardware/Software Interface

Adding the value of two registers



ANDS <Rdn>, <Rm>



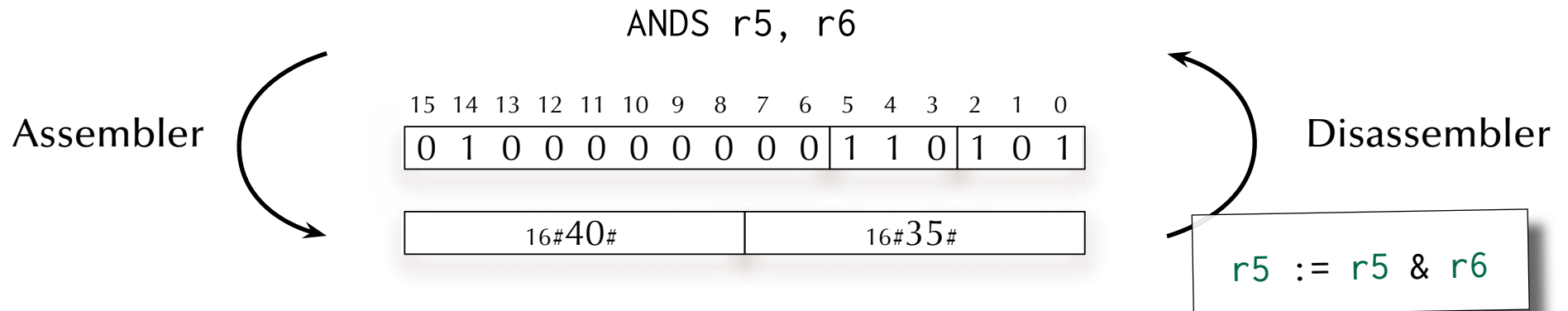
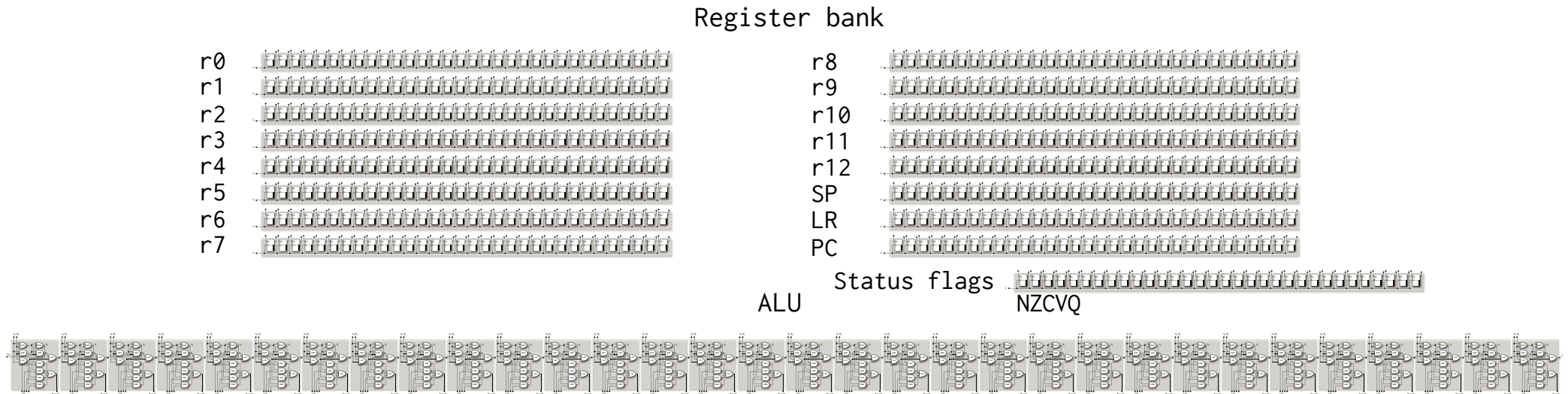
Op Code

Arguments



Hardware/Software Interface

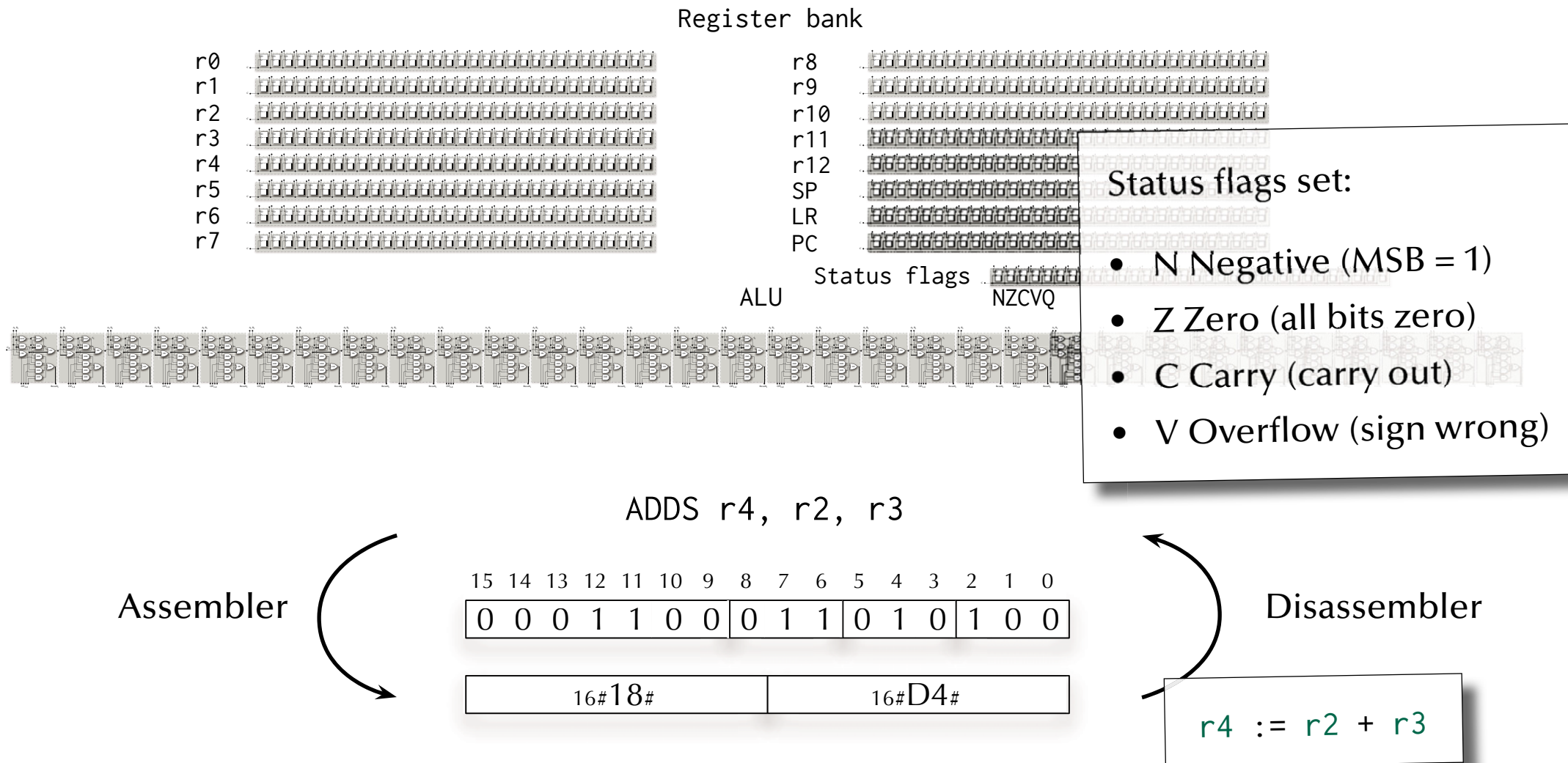
Adding the value of two registers





Hardware/Software Interface

Adding the value of two registers





Hardware/Software Interface

ARM v7-M 32 bit add instructions

add{s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}

adc{s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}

add{s}<c><q> {<Rd>}, <Rn>, #<const>

adc{s}<c><q> {<Rd>}, <Rn>, #<const>

qadd<c><q> {<Rd>}, <Rn>, <Rm>

adds r1, r4, r5

adcs r1, r4

add r1, #1

qadd r1, r4, r5

s: **sets the flags** based on the result

c: makes the command **conditional**. <c> can be EQ (equal), NE (not equal), CS (carry set), CC (carry clear), MI (minus), PL (plus), VS (overflow set), VC (overflow clear), HI (unsigned higher), LS (unsigned lower or same), GE (signed greater or equal), LT (signed less), GT (signed greater), LE (signed less or equal), AL (always)

q: **instruction width**. Can be .N for narrow (16 bit) or .W for wide (32 bit)

Rd, Rn, Rm: **any register**, incl. SP, LR and PC (with some restrictions). Result goes to Rn (if no Rd).

shift: value of Rm is **preprocessed** with LSL (logical shift left – fills zeros), LSR (logical shift right – fills zeros), ASR (arithmetic shift right – keeps sign) or ROR (rotate right) followed by the #number of bits to shift/rotate by. There is also a RRX (rotate right by one incl. carry flag)

const: an **immediate value** in the range 0..4095 directly or in the range 0..255 with rotation.



Hardware/Software Interface

ARM v7-M 32 bit add instructions

add{s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}

adc{s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}

add{s}<c><q> {<Rd>}, <Rn>, #<const>

adc{s}<c><q> {<Rd>}, <Rn>, #<const>

qadd<c><q> {<Rd>}, <Rn>, <Rm>

s: **sets the flags** based on the result

c: makes the command **conditional**. <c> can be EQ (equal), NE (not equal), CS (carry set), CC (carry clear), MI (minus), PL (plus), VS (overflow set), VC (overflow clear), HI (unsigned higher), LS (unsigned lower or same), GE (signed greater or equal), LT (signed less), GT (signed greater), LE (signed less or equal), AL (always)

q: **instruction width**. Can be .N for narrow (16 bit) or .W for wide (32 bit)

Rd, Rn, Rm: **any register**, incl. SP, LR and PC (with some restrictions). Result goes to Rn (if no Rd).

shift: value of Rm is **preprocessed** with LSL (logical shift left – fills zeros), LSR (logical shift right – fills zeros), ASR (arithmetic shift right – keeps sign) or ROR (rotate right) followed by the #number of bits to shift/rotate by. There is also a RRX (rotate right by one incl. carry flag)

const: an **immediate value** in the range 0..4095 directly or in the range 0..255 with rotation.

Any of those instructions
requires exactly *one* CPU cycle
(in terms of throughput).

“Reduced Instruction
Set Computing (RISC)”

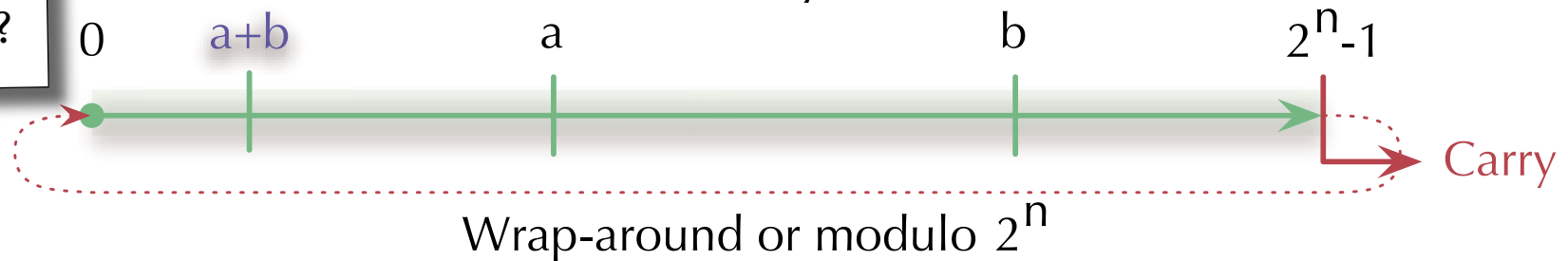


Hardware/Software Interface

Numeric CPU status flags

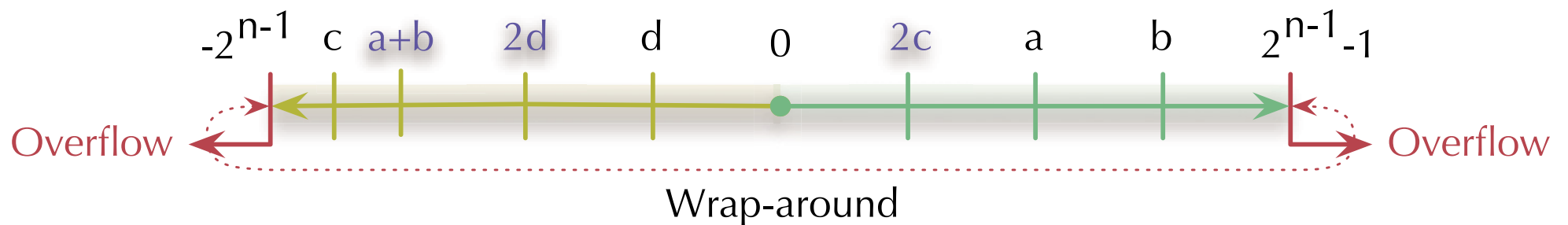
Which of those operations will set which flag?

Natural binary numbers

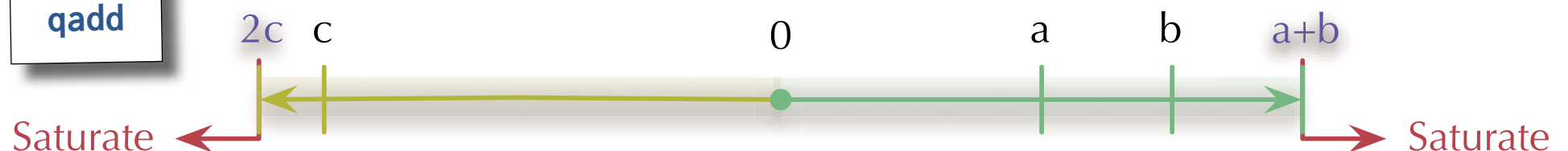


adds
adcs

2's complement binary numbers



qadd





Hardware/Software Interface

ARM v7-M 32 bit Addition, Subtraction instructions

add {s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}	; $Rd := Rn + Rm(\text{shifted})$
adc {s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}	; $Rd := Rn + Rm(\text{shifted}) + C$
add {s}<c><q> {<Rd>}, <Rn>, #<const>	; $Rd := Rn + \#<const>$
adc {s}<c><q> {<Rd>}, <Rn>, #<const>	; $Rd := Rn + \#<const> + C$
qadd <c><q> {<Rd>}, <Rn>, <Rm>	; $Rd := Rn + Rm$; saturated
sub {s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}	; $Rd := Rn - Rm(\text{shifted})$
sbc {s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}	; $Rd := Rn - Rm(\text{shifted}) - NOT(C)$
rsb {s}<c><q> {<Rd>}, <Rn>, <Rm> {,<shift>}	; $Rd := Rm(\text{shifted}) - Rn$
sub {s}<c><q> {<Rd>}, <Rn>, #<const>	; $Rd := Rn - \#<const>$
sbc {s}<c><q> {<Rd>}, <Rn>, #<const>	; $Rd := Rn - \#<const> - NOT(C)$
rsb {s}<c><q> {<Rd>}, <Rn>, #<const>	; $Rd := \#<const> - Rn$
qsub <c><q> {<Rd>}, Rn, Rm	; $Rd := Rn - Rm$; saturated

All instructions operate on 32bit wide numbers.

... versions for narrower numbers, as well as versions which operate on multiple narrower numbers in parallel exist as well.



Hardware/Software Interface

64 bit Addition, Subtraction

As your registers are 32 bit wide, you need two steps to add two 64 bit numbers in r3:r2, r5:r4 (with r2 and r4 being the lower 32 bits) to one 64 bit number in r1:r0:

```
adds  r0, r2, r4    ;  $r0 := r2 + r4$       add least significant words, set flags
adcs  r1, r3, r5    ;  $r1 := r3 + r5 + C$     add most significant words and carry bit
```

... and symmetrically if you need a 64 bit subtraction:

```
subs  r0, r2, r4    ;  $r0 := r2 - r4$       least significant words, set flags
sbcs  r1, r3, r5    ;  $r1 := r3 - r5 - NOT(C)$  most significant words and carry bit
```



Hardware/Software Interface

ARM v7-M 32 bit Boolean (bit-wise) instructions

and{s}<c><q> {<Rd>,} <Rn>, <Rm> {,<shift>} ; $Rd := Rn \wedge Rm^{shifted}$
bic{s}<c><q> {<Rd>,} <Rn>, <Rm> {,<shift>} ; $Rd := Rn \wedge \overline{Rm^{shifted}}$
orr{s}<c><q> {<Rd>,} <Rn>, <Rm> {,<shift>} ; $Rd := Rn \vee Rm^{shifted}$
orn{s}<c><q> {<Rd>,} <Rn>, <Rm> {,<shift>} ; $Rd := Rn \vee \overline{Rm^{shifted}}$
eor{s}<c><q> {<Rd>,} <Rn>, <Rm> {,<shift>} ; $Rd := Rn \oplus Rm^{shifted}$
and{s}<c><q> {<Rd>,} <Rn>, #<const> ; $Rd := Rn \wedge const$
bic{s}<c><q> {<Rd>,} <Rn>, #<const> ; $Rd := Rn \wedge \overline{const}$
orr{s}<c><q> {<Rd>,} <Rn>, #<const> ; $Rd := Rn \vee const$
orn{s}<c><q> {<Rd>,} <Rn>, #<const> ; $Rd := Rn \vee \overline{const}$
eor{s}<c><q> {<Rd>,} <Rn>, #<const> ; $Rd := Rn \oplus const$
cmp<c><q> <Rn>, <Rm> {,<shift>} ; $(Rn - Rm^{shifted}) \rightarrow Flags$
cmn<c><q> <Rn>, <Rm> {,<shift>} ; $(Rn + Rm^{shifted}) \rightarrow Flags$
tst<c><q> <Rn>, <Rm> {,<shift>} ; $(Rn \wedge Rm^{shifted}) \rightarrow Flags$
teq<c><q> <Rn>, <Rm> {,<shift>} ; $(Rn \oplus Rm^{shifted}) \rightarrow Flags$
cmp<c><q> <Rn>, #<const> ; $(Rn - const) \rightarrow Flags$
cmn<c><q> <Rn>, #<const> ; $(Rn + const) \rightarrow Flags$
tst<c><q> <Rn>, #<const> ; $(Rn \wedge const) \rightarrow Flags$
teq<c><q> <Rn>, #<const> ; $(Rn \oplus const) \rightarrow Flags$

This exhausts
the simple
ALU from
chapter 1 ...

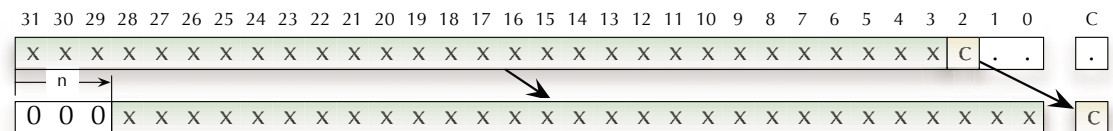


Hardware/Software Interface

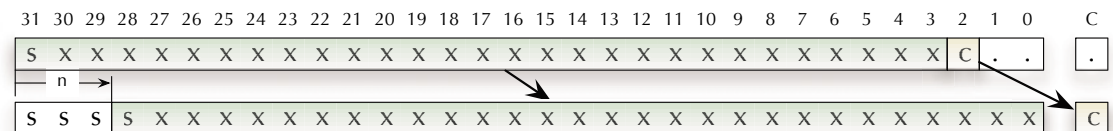
ARM v7-M Move data inside the CPU

mov{s}<c><q> <Rd>, <Rm> ; $Rd := Rm$
mov{s}<c><q> <Rd>, #<const> ; $Rd := const$

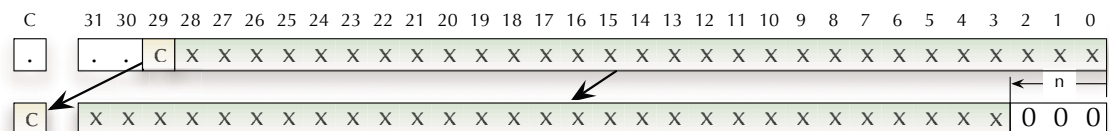
lsr{s}<c><q> <Rd>, <Rm>, #<n>
lsr{s}<c><q> <Rd>, <Rm>, <Rs>



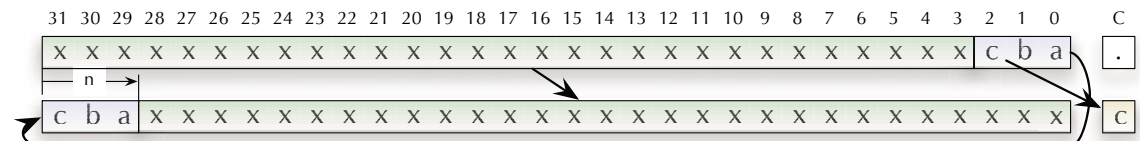
asr{s}<c><q> <Rd>, <Rm>, #<n>
asr{s}<c><q> <Rd>, <Rm>, <Rs>



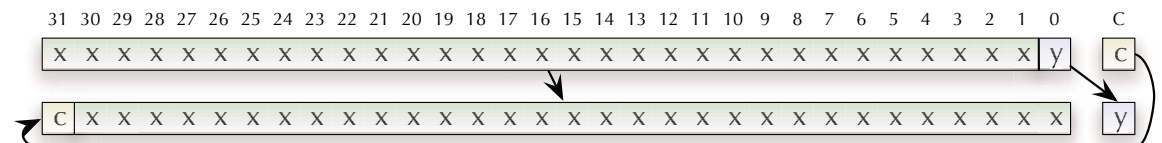
lsl{s}<c><q> <Rd>, <Rm>, #<n>
lsl{s}<c><q> <Rd>, <Rm>, <Rs>



ror{s}<c><q> <Rd>, <Rm>, #<n>
ror{s}<c><q> <Rd>, <Rm>, <Rs>



rrx{s}<c><q> <Rd>, <Rm>





Hardware/Software Interface

ARM v7-M Move data inside the CPU

mov{s}<c><q> <Rd>, <Rm>

; $R_d := R_m$

mov{s}<c><q> <Rd>, #<const>

; $R_d := \text{const}$

If this is numbers then ...

lsr{s}<c><q> <Rd>, <Rm>, #<n>

lsr{s}<c><q> <Rd>, <Rm>, <Rs>

;

asr{s}<c><q> <Rd>, <Rm>, #<n>

asr{s}<c><q> <Rd>, <Rm>, <Rs>

;

lsl{s}<c><q> <Rd>, <Rm>, #<n>

lsl{s}<c><q> <Rd>, <Rm>, <Rs>

;

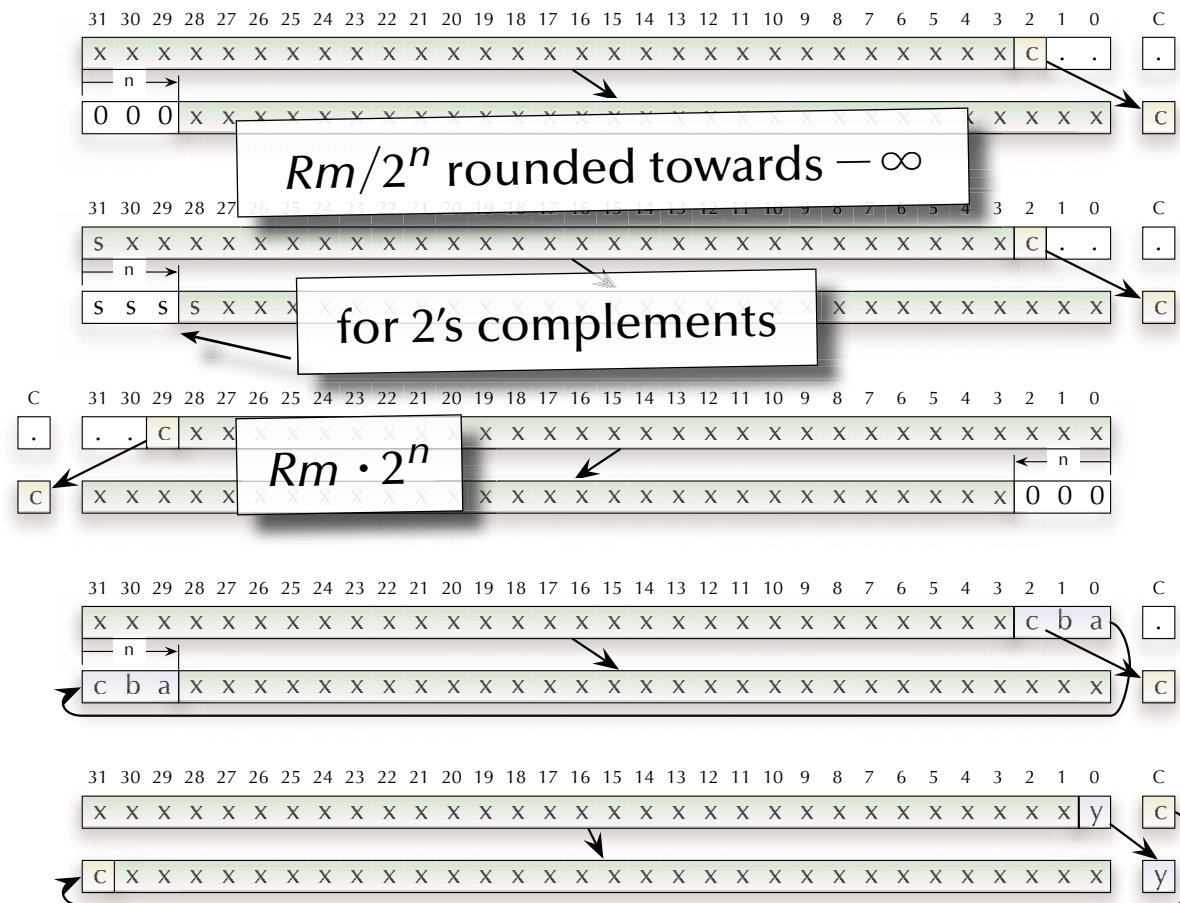
ror{s}<c><q> <Rd>, <Rm>, #<n>

ror{s}<c><q> <Rd>, <Rm>, <Rs>

;

rrx{s}<c><q> <Rd>, <Rm>

;





Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$$e := a + b - 2 * c$$

assuming all types are 32 bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.



Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$e := a + b - 2 * c$

assuming all types are 32 bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.

```
add    r5, r1, r2
lsl    r6, r3, #1    ; you could also write: mov r6, r3, lsl #1
sub    r4, r5, r6
```

We need temporary storage (r5, r6) in the process as we didn't want to overwrite the original values. Yet the total number of registers is always limited.



Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$e := a + b - 2 * c$

assuming all types are 32bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.

```
add    r5, r1, r2
lsl    r6, r3, #1    ; you could also write: mov r6, r3, lsl #1
sub    r4, r5, r6
```

We need temporary storage (r5, r6) in the process as we didn't want to overwrite the original values. Yet the total number of registers is always limited.

How about we assume that values are no longer needed after this expression:



Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$e := a + b - 2 * c$

assuming all types are 32bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.

```
add    r5, r1, r2
lsl    r6, r3, #1    ; you could also write: mov r6, r3, lsl #1
sub    r4, r5, r6
```

We need temporary storage (r5, r6) in the process as we didn't want to overwrite the original values. Yet the total number of registers is always limited.

How about we assume that values are no longer needed after this expression:

```
add    r1, r1, r2
lsl    r3, r3, #1
sub    r4, r1, r3
```

... your compiler will know when such side-effects are ok and when not.

☞ Any overflows?



Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$e := a + b - 2 * c$

We need to check results after each step:

```
adds  r1, r1, r2    ; need to check overflow flag
lsl   r3, r3, #1     ; need to check that the sign did not change
subs  r4, r1, r3     ; need to check overflow flag again
```

☞ We don't have the means yet to branch off into different actions in case things go bad ... to come soon.



Hardware/Software Interface

Simple arithmetic inside the CPU

Calculate:

$e := a + b - 2 * c$

We need to check results after each step:

```
adds  r1, r1, r2    ; need to check overflow flag
lsl   r3, r3, #1     ; need to check that the sign did not change
subs  r4, r1, r3     ; need to check overflow flag again
```

☞ We don't have the means yet to branch off into different actions in case things go bad ... to come soon.

Or we use saturation arithmetic and live with the error:

```
qadd  r1, r1, r2
qadd  r3, r3, r3
qsub  r4, r1, r3
```

☞ If we know we need to carry on either way, this at least minimizes the local errors.



Hardware/Software Interface

Cortex-M4 Address Space

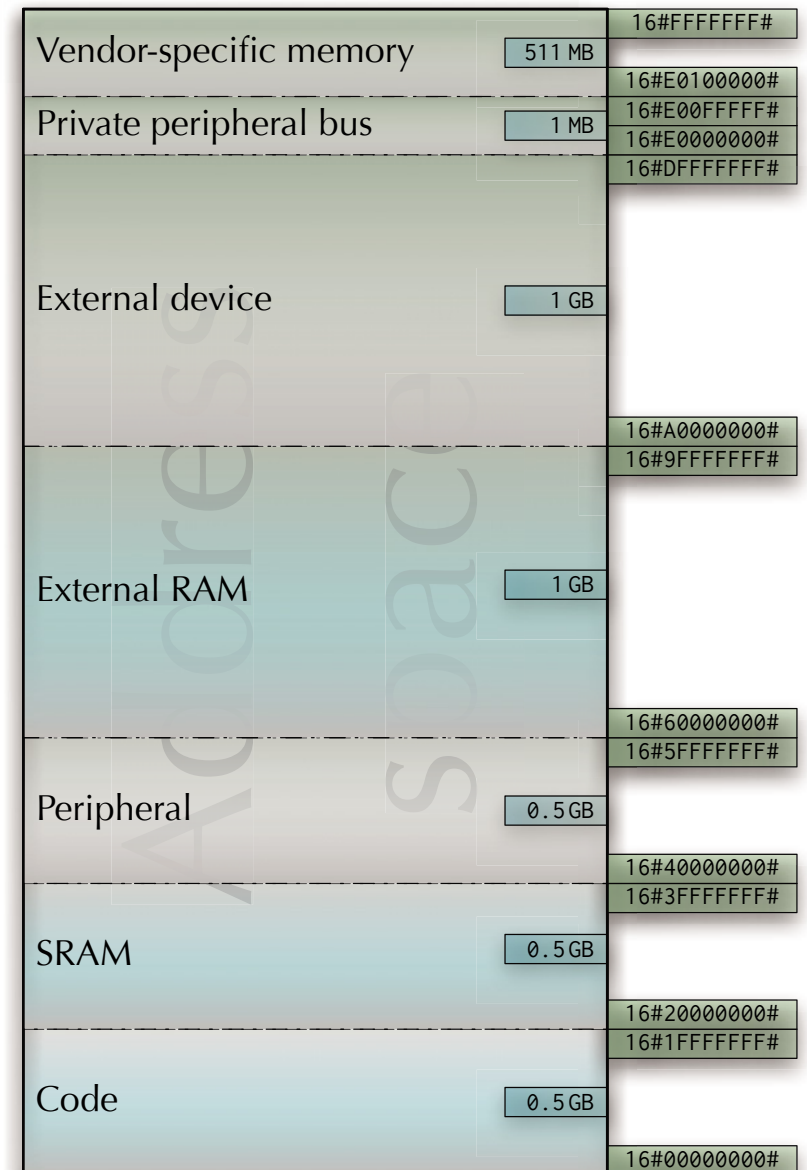
Your CPU has 32 bit of address space

➡ 4 GB

... address space does not equate to physical memory!

Not all memory is equal: Some memory ...

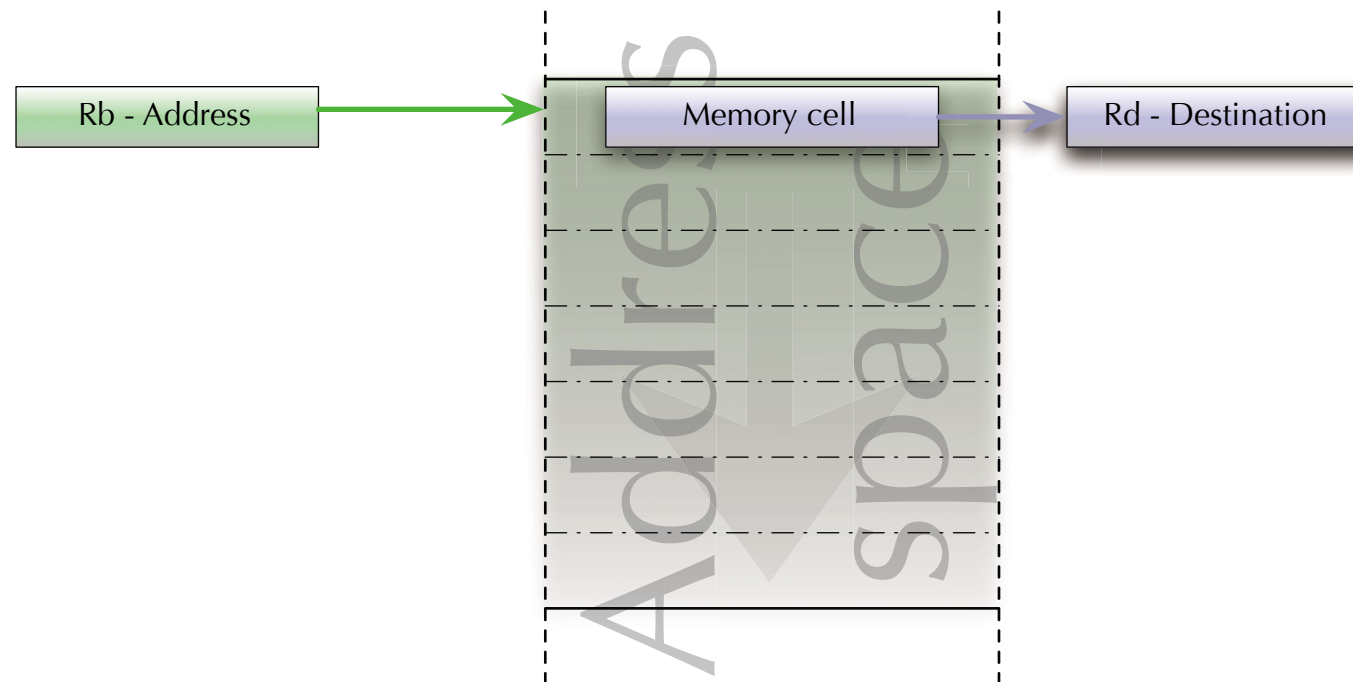
- ... can be **executed**
- ... can be **written to or read from or both**
- ... has **side-effects** (coffee cups fall over)
- ... has **strictly-ordered** access
- ... **does not physically exist**





Hardware/Software Interface

ARM v7-M Copy data in and out of the CPU

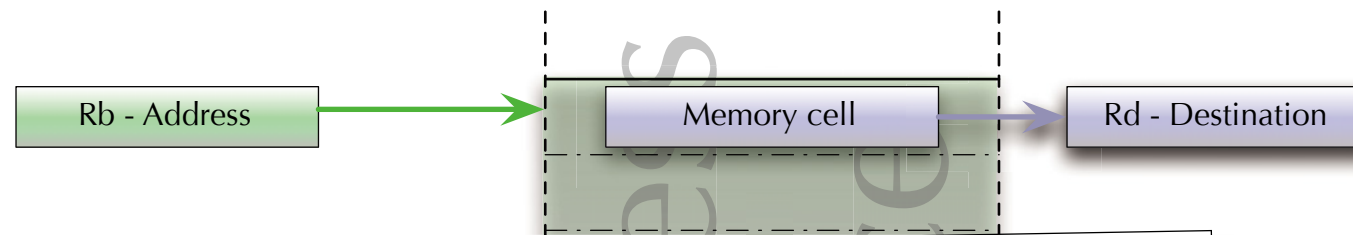


In its most basic form the value of a register is interpreted as an **address** and the **memory content** there is loaded into another register.



Hardware/Software Interface

ARM v7-M Copy data in and out of the CPU



Yet: most data is structured.

... like a group of local variables, a record, an array and any combination of the above ...

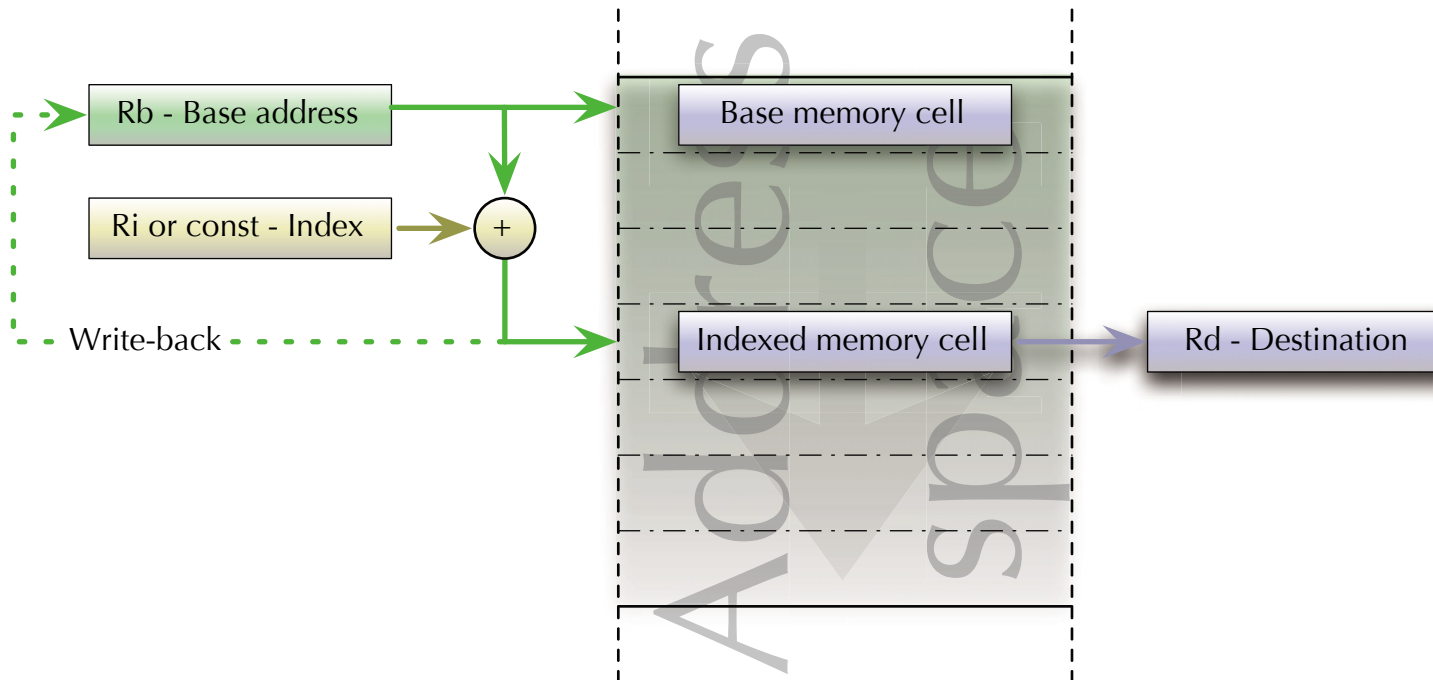
☞ How to read an entry in an array/record?

In its most basic form the value of a register is interpreted as an **address** and the **memory content** there is loaded into another register.



Hardware/Software Interface

ARM v7-M Copy data in and out of the CPU

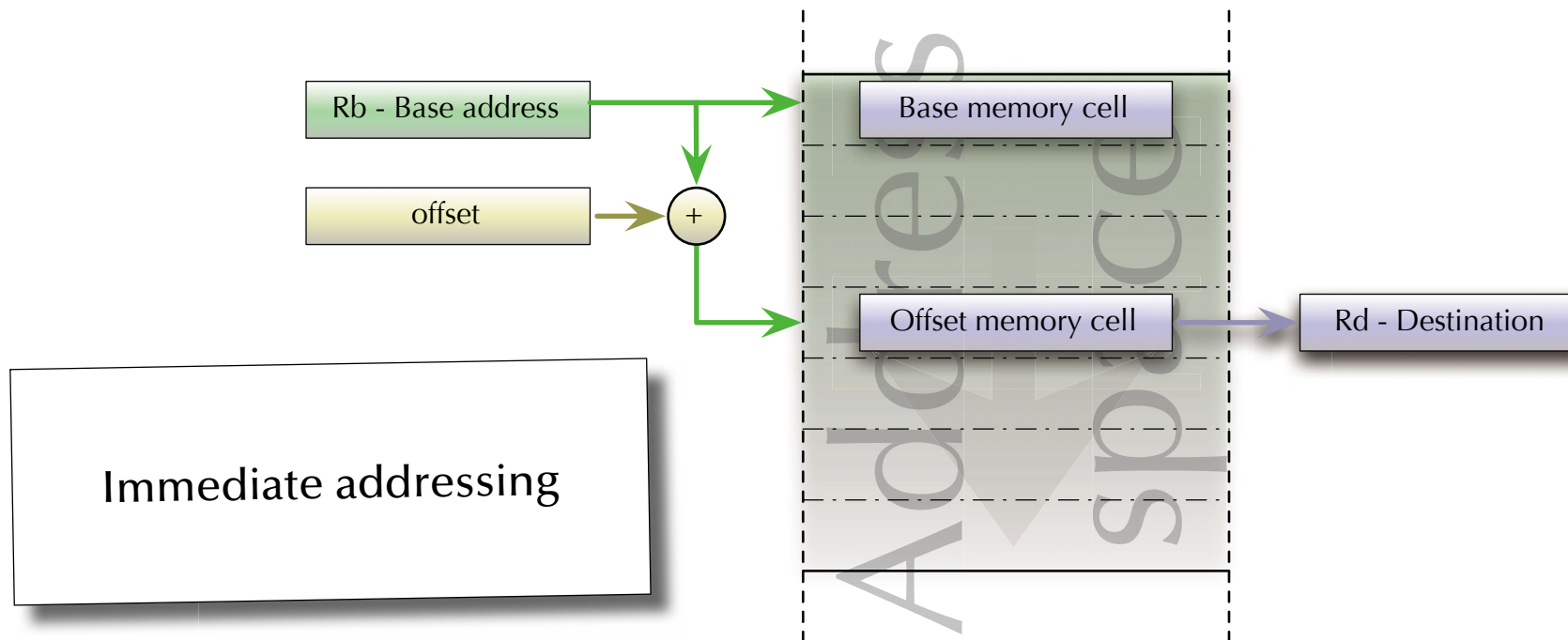


Most copy operations between CPU and memory follow this basic scheme.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb> {, #+/-<offset>}]
str<c><q> <Rs>, [<Rb> {, #+/-<offset>}]

ldr r1, [r4]

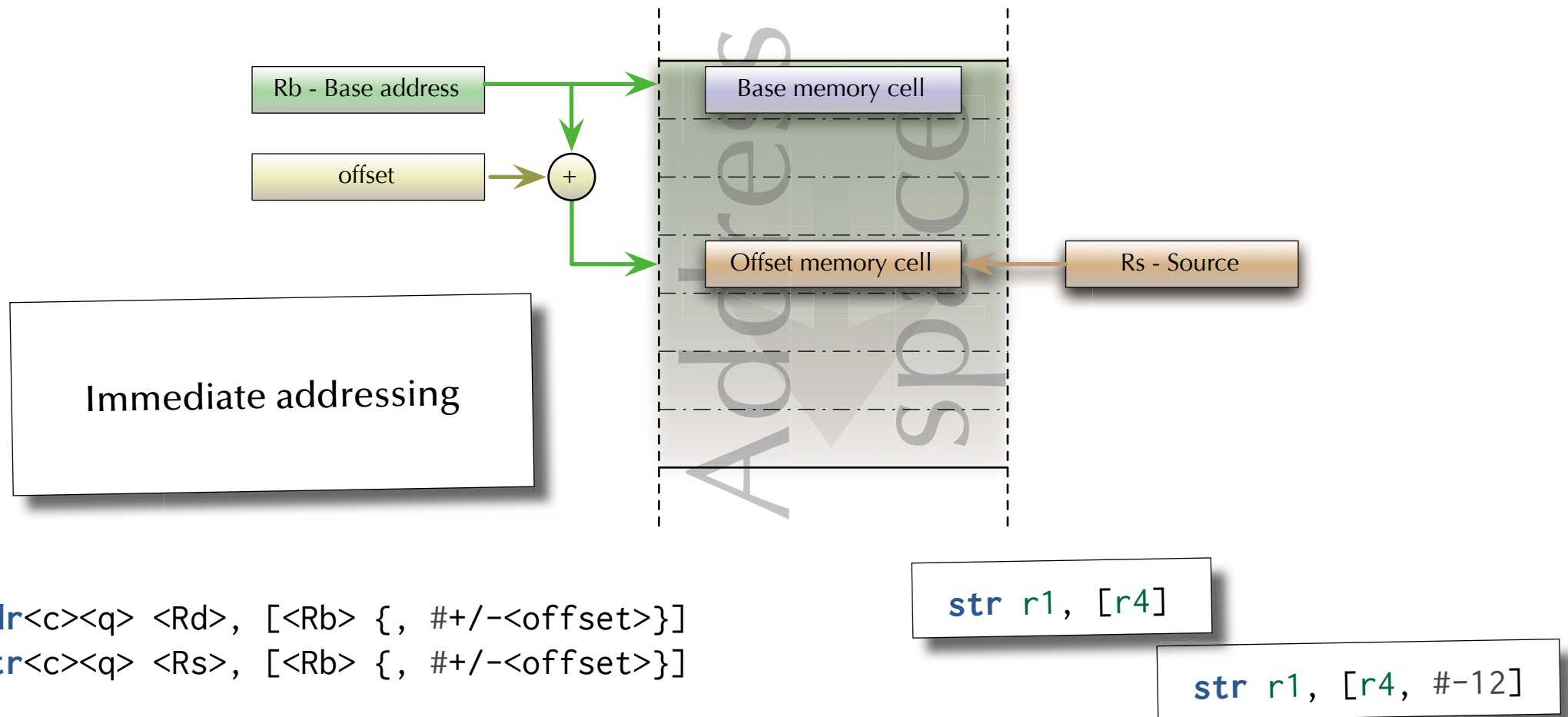
ldr r1, [r4, #8]

Reads from a potentially offset memory cell with a base register address.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

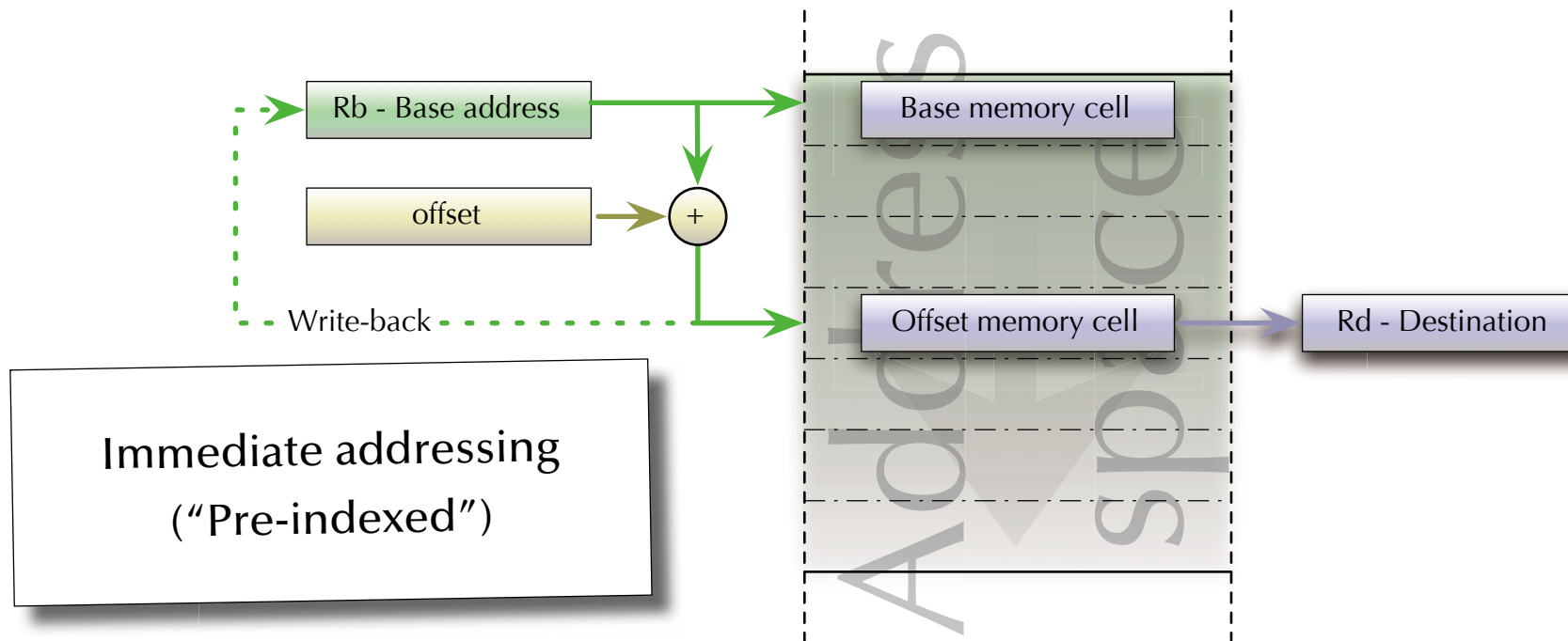


Writes to a potentially offset memory cell with a base register address.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb>, #+/-<offset>]!
str<c><q> <Rs>, [<Rb>, #+/-<offset>]!

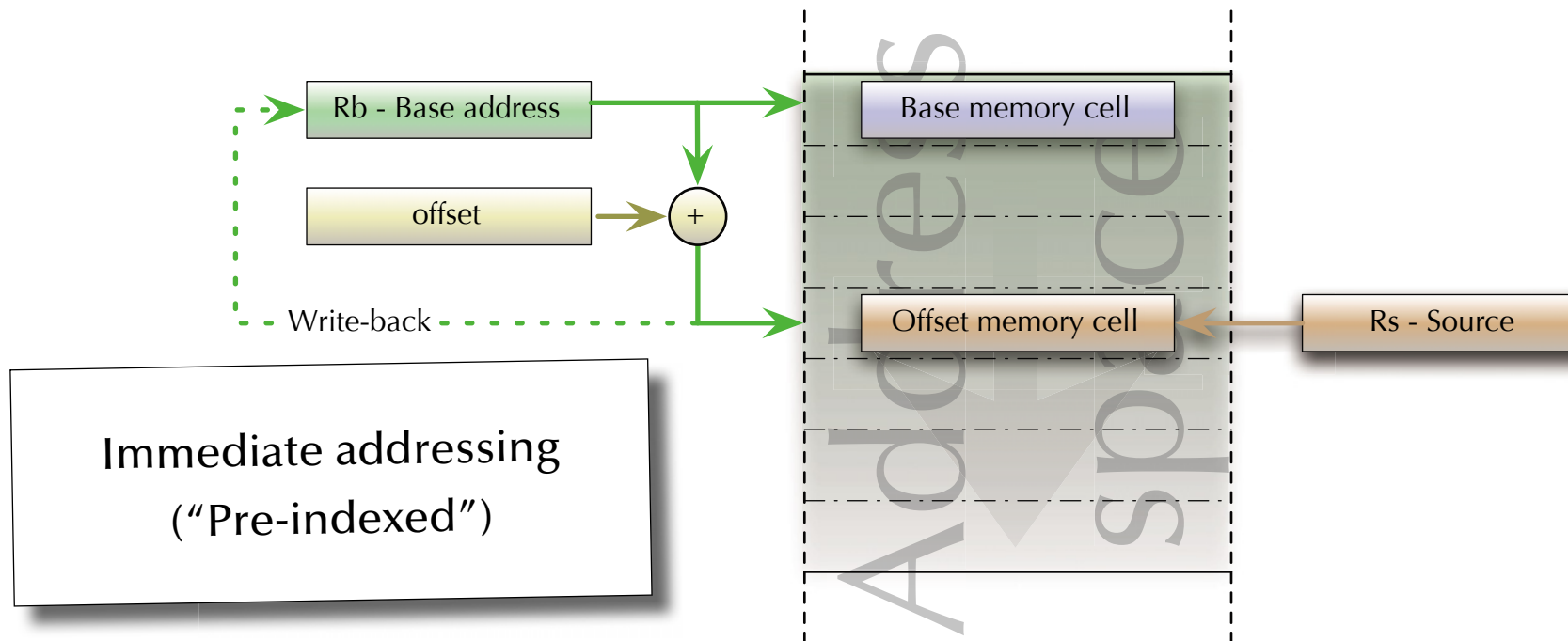
ldr r1, [r4, #8]!

Reads from an offset memory cell with a base register address and writes the offset address back into the original base register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb>, #+/-<offset>]!
str<c><q> <Rs>, [<Rb>, #+/-<offset>]!

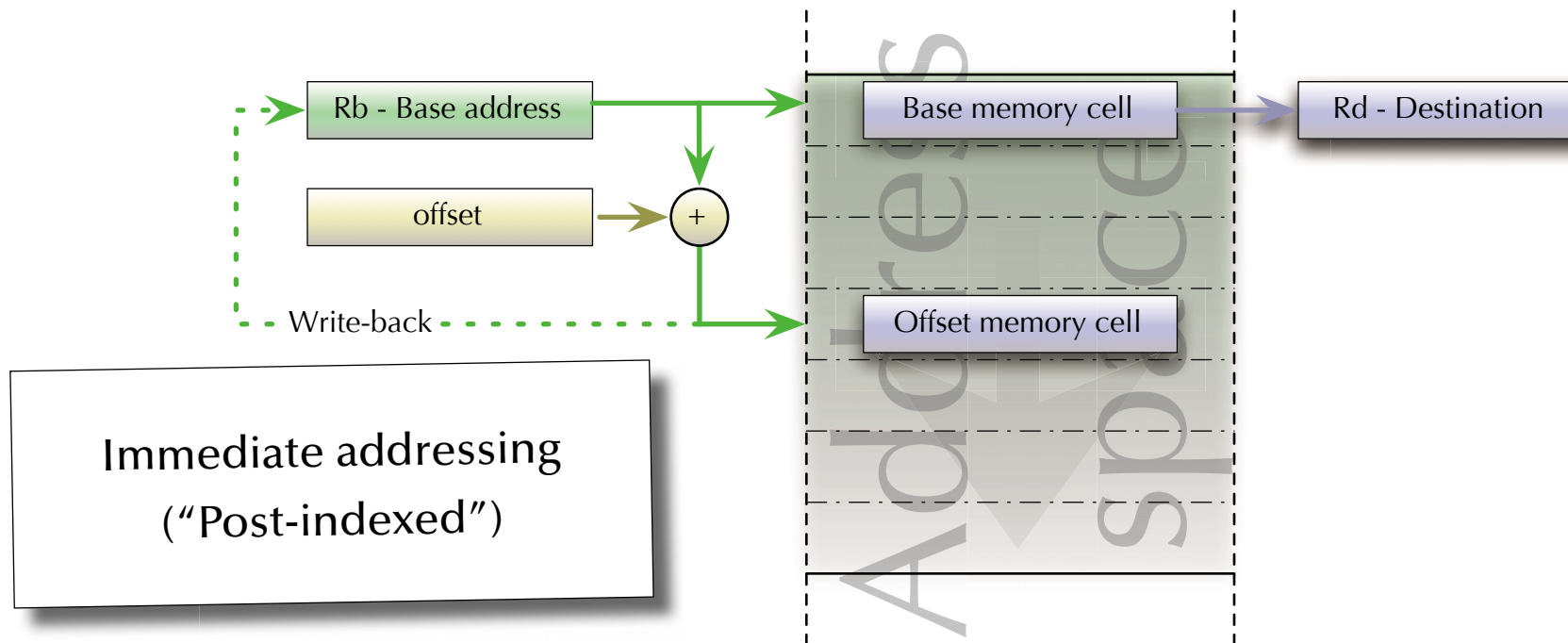
str r1, [r4, #-12]!

Writes to an offset memory cell with a base register address and writes the offset address back into the original base register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb>], #+/-<offset>
str<c><q> <Rs>, [<Rb>], #+/-<offset>

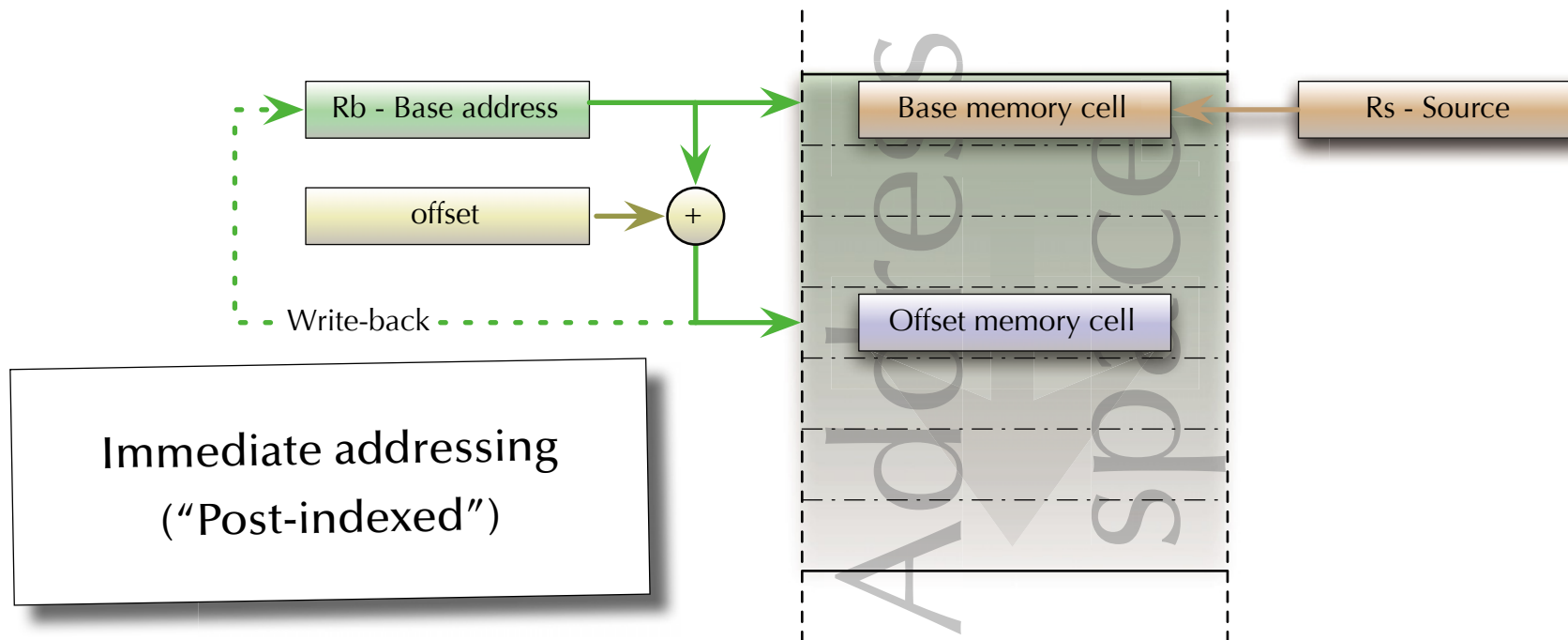
ldr r1, [r4], #8

Reads from a memory cell with a base register address
and writes the offset address back into the original base register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb>], #+/-<offset>
str<c><q> <Rs>, [<Rb>], #+/-<offset>

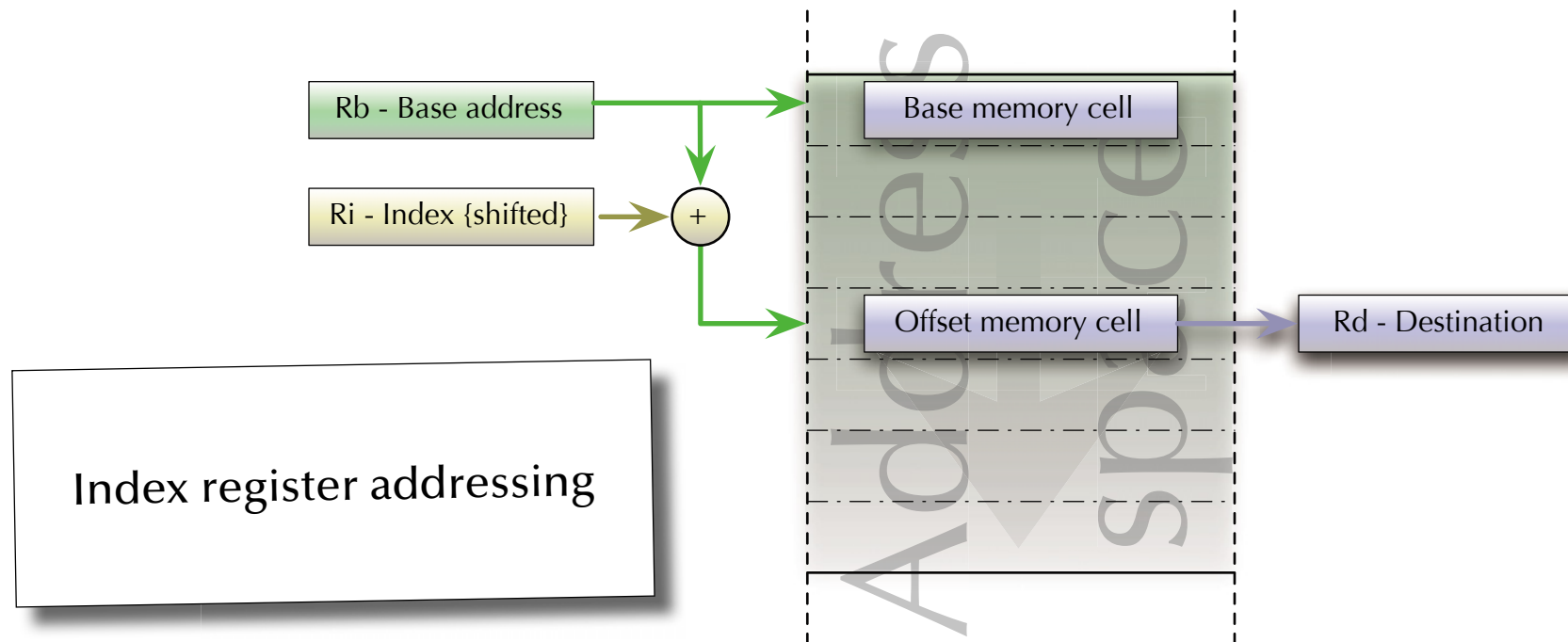
str r1, [r4], #8

Writes to a memory cell with a base register address
and writes the offset address back into the original base register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr<c><q> <Rd>, [<Rb>, <Ri> {, LSL #<shift>}]

str<c><q> <Rs>, [<Rb>, <Ri> {, LSL #<shift>}]

ldr r1, [r4, r3]

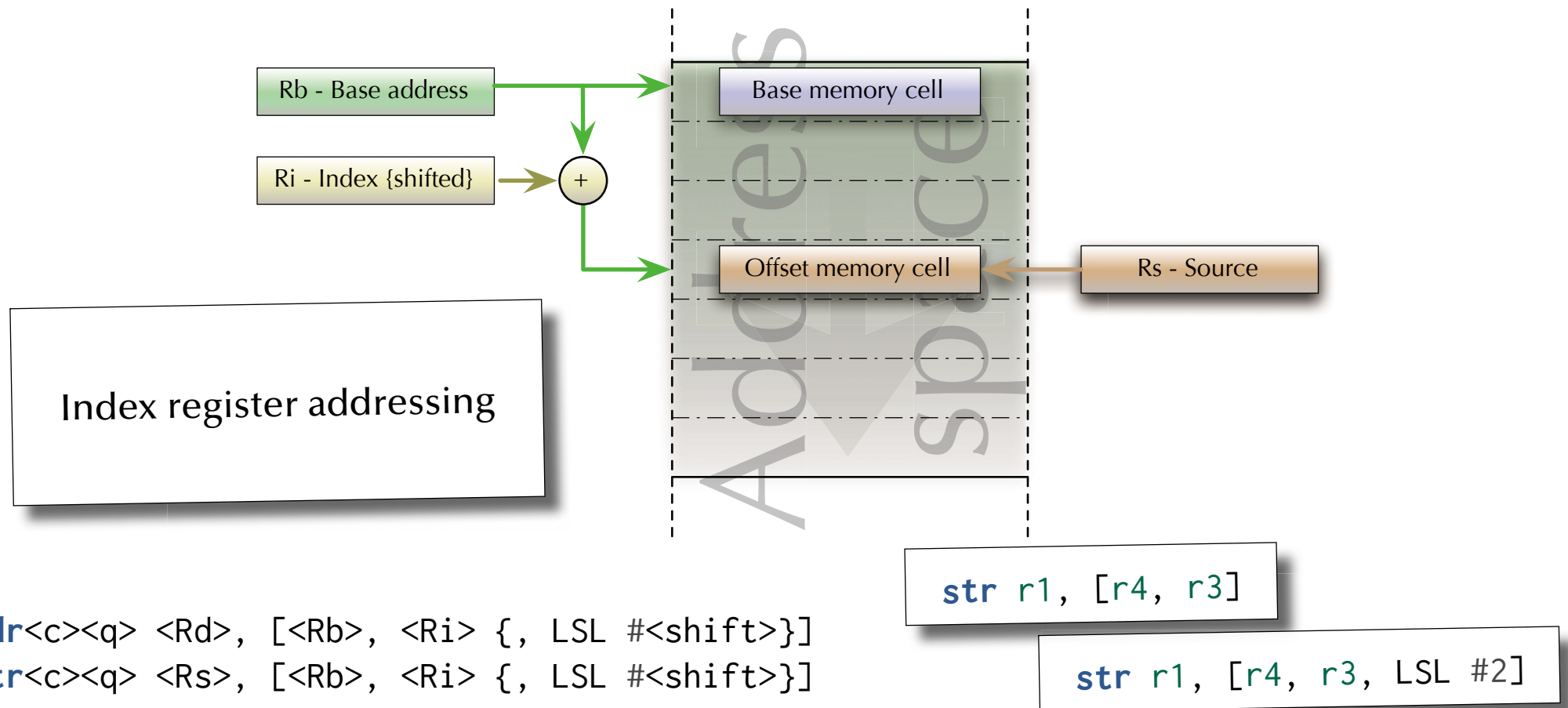
ldr r1, [r4, r3, LSL #2]

Reads from a memory cell with a base register address plus a potentially shifted index register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

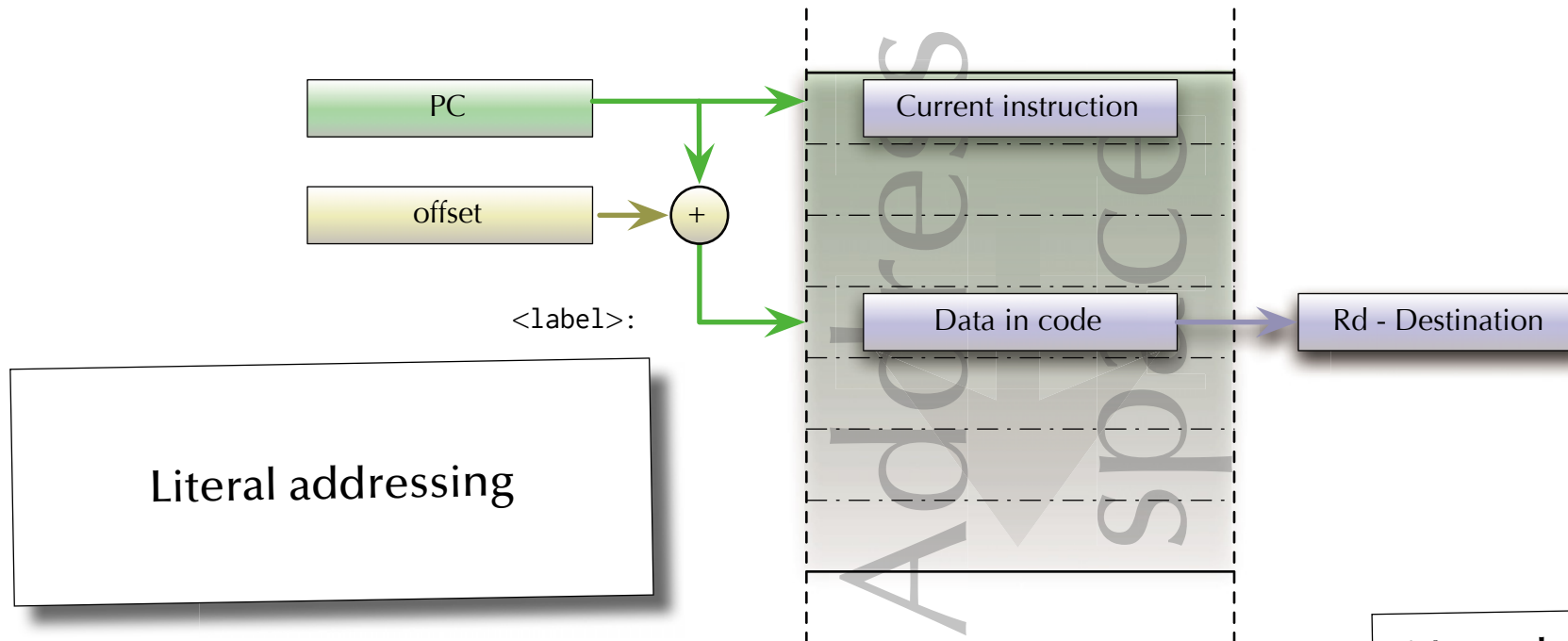


Writes to a memory cell with a base register address plus a potentially shifted index register.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



Note there is no store version.

ldr<c><q> <Rd>, <label>

ldr<c><q> <Rd>, [PC, #+/-<offset>]

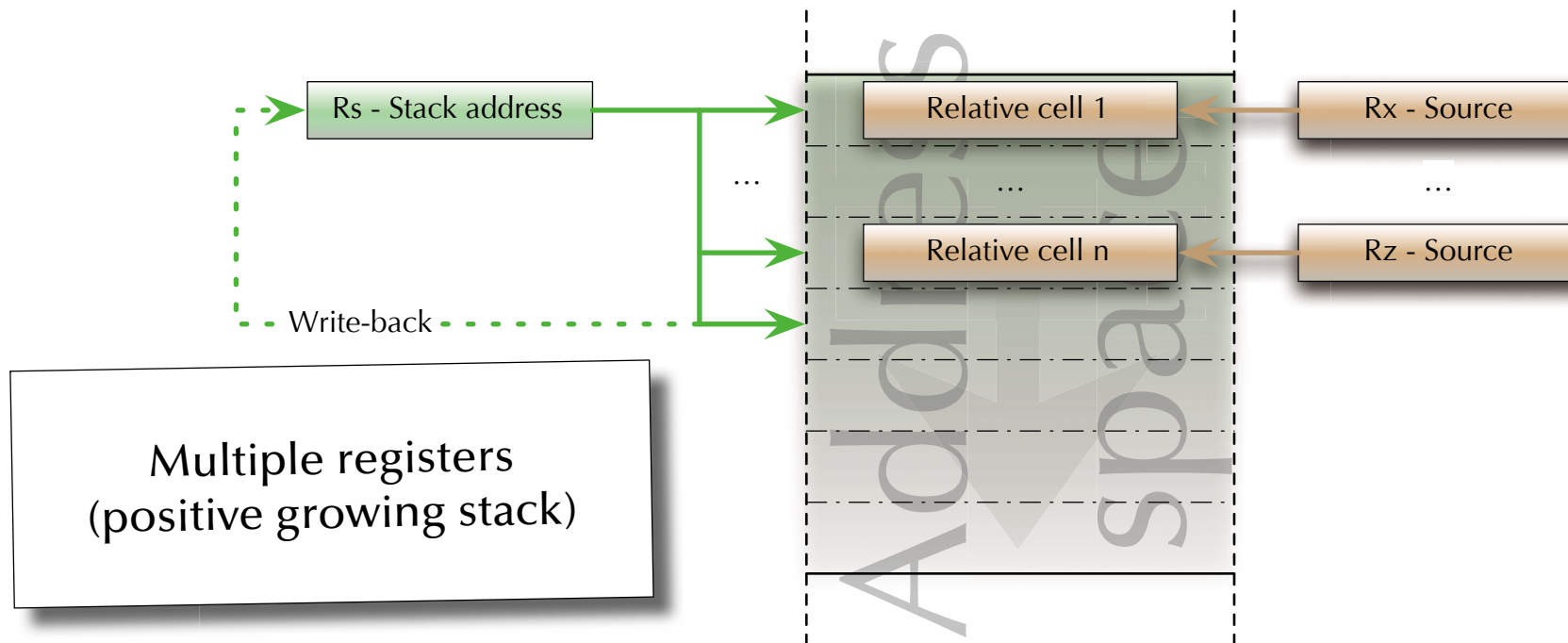
Reads from a data area embedded into the code section.

ldr r1, data



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



stmia<c><q> <Rs>{!}, <registers>
ldmdb<c><q> <Rs>{!}, <registers>

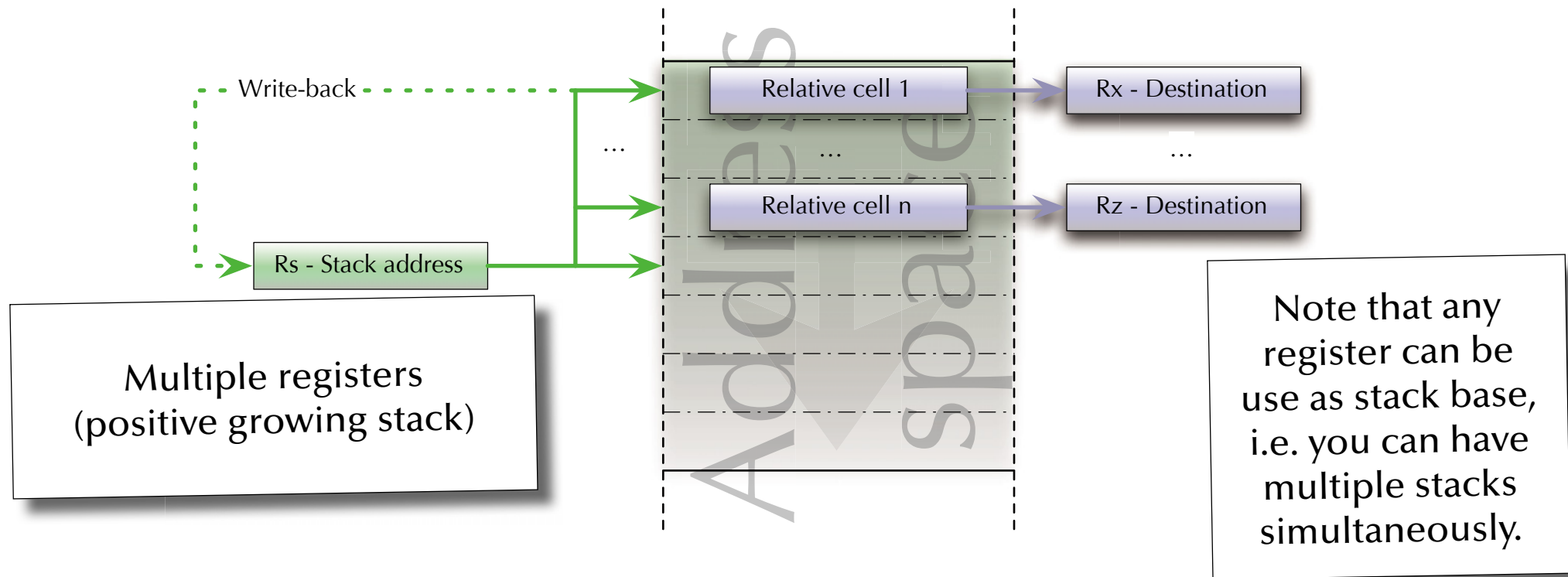
stmia r9!, {r1, r3, r4, fp}

Stores multiple registers into sequential memory addresses.
Stores “increment after” and loads “decrement before”.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



```
stmia<c><q> <Rs>{!}, <registers>  
ldmdb<c><q> <Rs>{!}, <registers>
```

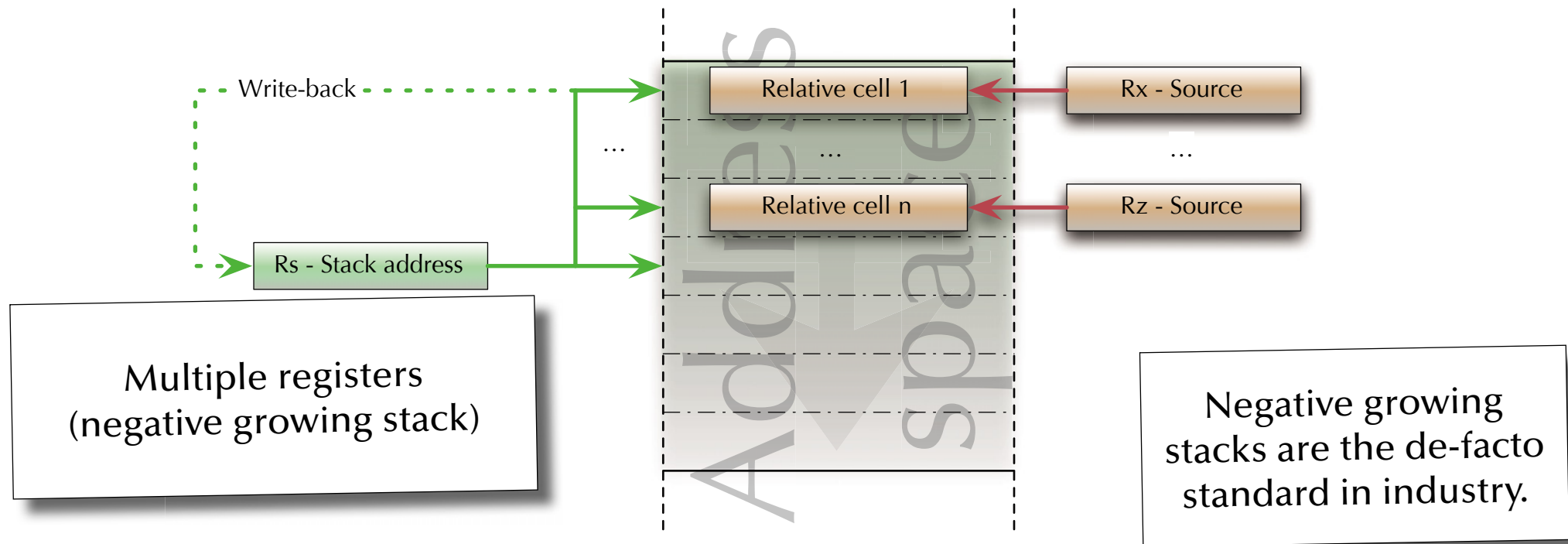
```
ldmdb r9!, {r1, r3, r4, fp}
```

Reads multiple registers from sequential memory addresses.
Stores “increment after” and loads “decrement before”.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



```
stmdb<c><q> <Rs>{!}, <registers>  
ldmia<c><q> <Rs>{!}, <registers>
```

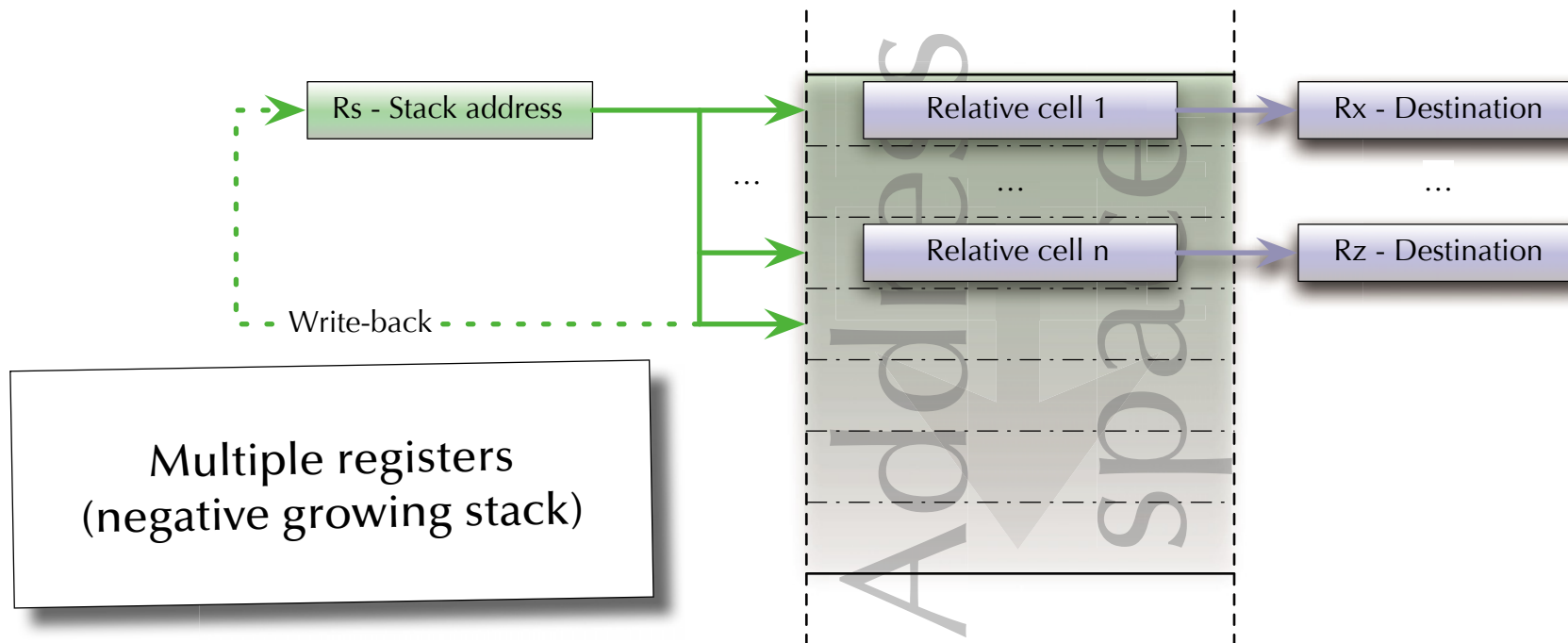
```
stmdb SP!, {r1, r3, r4, fp}
```

Stores multiple registers to sequential memory addresses.
Stores “decrement before” and loads “increment after”.



Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



```
stmdb<c><q> <Rs>{!}, <registers>  
ldmia<c><q> <Rs>{!}, <registers>
```

```
ldmia SP!, {r1, r3, r4, fp}
```

Reads multiple registers from sequential memory addresses.
Stores “decrement before” and loads “increment after”.



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2 * c$

but now a , b , c and e are stored in memory, relative to an address stored in FP (“Frame Pointer”):
 a is held at $[fp - 12]$, b at $[fp - 16]$, c at $[fp - 20]$ and e at $[fp - 24]$

In order to do arithmetic we need to load those values into the CPU first and afterwards we need to store the result in memory:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
add    r1, r1, r2
ldr    r2, [fp, #-20]
lsl    r2, r2, #1
sub    r1, r1, r2
str    r1, [fp, #-24]
```

☞ Notice that this time we only used two registers.



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2*c$

Or in saturation arithmetic:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
qadd   r1, r1, r2
ldr    r2, [fp, #-20]
qadd   r2, r2, r2
qsub   r1, r1, r2
str    r1, [fp, #-24]
```



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2 * c$

Or with overflow checks:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
adds   r1, r1, r2      ; need to check overflow flag
ldr    r2, [fp, #-20]
lsl    r2, r2, #1       ; need to check that the sign did not change
subs   r1, r1, r2       ; need to check overflow flag
str    r1, [fp, #-24]
```

☞ It's time we learn about branching off into alternative execution paths.



Hardware/Software Interface

ARM v7-M Branch instructions

b<c><q> <label> ; if c then PC := label
bl<c> <label> ; if c then LR := PC_next; PC := label
bx<c> <Rm> ; if c then PC := Rm
blx<c><q> <Rm> ; if c then LR := PC_next; PC := Rm
cbz<q> <Rn>, <label> ; if Rn = 0 then PC := label
cbnz<q> <Rn>, <label> ; if Rn != 0 then PC := label

<c>	Meanings	Flags
eq	Equal	Z = 1
ne	Not equal	Z = 0
cs, hs	Carry set, Unsigned higher or same	C = 1
cc, lo	Carry clear, Unsigned lower	C = 0
mi	Minus, Negative	N = 1
pl	Plus, Positive or zero	N = 0
vs	Overflow	V = 1
vc	No overflow	V = 0
hi	Unsigned higher	C = 1 $\wedge$ Z = 0
ls	Unsigned lower or same	C = 0 $\vee$ Z = 1
ge	Signed greater or equal	N = Z
lt	Signed less	N $\neq$ Z
gt	Signed greater	Z = 0 $\wedge$ N = V
le	Signed less or equal	Z = 1 $\vee$ N $\neq$ V
al, <none>	Always	any



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2 * c$

Or with overflow checks:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
adds   r1, r1, r2
bvs    Overflow           ; branch if overflow is set
ldr    r2, [fp, #-20]
adds   r2, r2, r2
bvs    Overflow           ; branch if overflow is set
subs   r1, r1, r2
bvs    Overflow           ; branch if overflow is set
str    r1, [fp, #-24]
```

...

Overflow:

```
svc    #5                ; call the operating system or runtime environment with #5
                        ; (assuming that #5 indicates an overflow situation)
```



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2 * c$

Or with overflow checks:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
adds   r1, r1, r2
bvs    Overflow           ; branch if overflow is set
ldr    r2, [fp, #-20]
adds   r2, r2, r2
bvs    Overflow           ; branch if overflow is set
subs   r1, r1, r2
bvs    Overflow           ; branch if overflow is set
str    r1, [fp, #-24]
```

...

Overflow:

```
svc    #5                ; call the operating system or runtime environment with #5
                        ; (assuming that #5 indicates an overflow situation)
```

... but how do we know
where this happened or how
to continue operations?



Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

$e := a + b - 2 * c$

Or with overflow checks:

```
ldr    r1, [fp, #-12]
ldr    r2, [fp, #-16]
adds   r1, r1, r2
blvs   Overflow           ; branch if overflow is set; keep next location in LR
ldr    r2, [fp, #-20]
adds   r2, r2, r2
blvs   Overflow           ; branch if overflow is set; keep next location in LR
subs   r1, r1, r2
blvs   Overflow           ; branch if overflow is set; keep next location in LR
str    r1, [fp, #-24]
```

...

Overflow:

```
...           ; ... for example writing a log entry with location
bx      lr     ; resume operations - assuming the above did not change LR
```



Hardware/Software Interface

ARM v7-M Essential multiplications and divisions

32 bit to 32 bit

mul {s}<c><q>	{<Rd>, } <Rn>, <Rm>	; $Rd := (Rn * Rm)$
mla <c>	<Rd>, <Rn>, <Rm>, <Ra>	; $Rd := Ra + (Rn * Rm)$
mls <c>	<Rd>, <Rn>, <Rm>, <Ra>	; $Rd := Ra - (Rn * Rm)$
udiv <c>	<Rd>, <Rn>, <Rm>	; $Rd := \text{unsigned } (Rn / Rm); \text{ rounded towards } 0$
sdiv <c>	<Rd>, <Rn>, <Rm>	; $Rd := \text{signed } (Rn / Rm); \text{ rounded towards } 0$

32 bit to 64 bit

umull <c>	<RdLo>, <RdHi>, <Rn>, <Rm>	; $RdHi:RdLo := \text{unsigned } (Rn * Rm)$
umlal <c><q>	<RdLo>, <RdHi>, <Rn>, <Rm>	; $RdHi:RdLo := \text{unsigned } (RdHi:RdLo + (Rn * Rm))$
smull <c>	<RdLo>, <RdHi>, <Rn>, <Rm>	; $RdHi:RdLo := \text{signed } (Rn * Rm)$
smlal <c>	<RdLo>, <RdHi>, <Rn>, <Rm>	; $RdHi:RdLo := \text{signed } (RdHi:RdLo + (Rn * Rm))$

... versions for narrower numbers, as well as versions which operate on multiple narrower numbers in parallel exist as well.



Hardware/Software Interface

Straight power

Calculate:

$c := a^b$

```
mov    r1, #7           ; a
mov    r2, #11          ; b ; has to be non-negative
mov    r3, #1           ; c
```

power:

```
cbz    r2, end_power    ; exponent zero?
mul    r3, r1
sub    r2, #1
b      power
```

end_power:

```
nop                ; c = a ^ b
```

$$7^{11} = 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7 \cdot 7$$

How many iterations?
How many cycles?



Hardware/Software Interface

More power

Calculate:

$c := a^b$

```
mov    r1, #7           ; a
mov    r2, #11          ; b ; has to be non-negative
mov    r3, #1           ; c
mov    r4, r1           ; base a to the powers of two, starting with a ^ 1
```

power:

```
cbz    r2, end_power    ; exponent zero?
tst    r2, #0b1         ; right-most bit of exponent set?
beq    skip            ; skip this power if not
mul    r3, r4           ; multiply the current power into result
```

skip:

```
mul    r4, r4           ; calculate next power
lsr    r2, #1           ; divide exponent by 2
b      power
```

end_power:

```
nop                    ; c = a ^ b
```

$$7^{11} = 7^8 \cdot 7^2 \cdot 7^1$$

How many iterations?
How many cycles?



Hardware/Software Interface

Table based branching

tbb<c><q> [<Rn>, <Rm>] ; for tables of offset bytes (8bit)

tbh<c><q> [<Rn>, <Rm>, **lsl** #1] ; for tables of offset halfwords (16bit)

Common usage for byte (8bit) tables

tbb [PC, Ri] ; PC is base of branch table, Ri is index

Branch_Table:

.byte (Case_A - Branch_Table)/2 ; Case_A 8 bit offset

.byte (Case_B - Branch_Table)/2 ; Case_B 8 bit offset

.byte (Case_C - Branch_Table)/2 ; Case_C 8 bit offset

.byte 0x00 ; Padding to re-align with halfword boundaries

Case_A:

... ; any instruction sequence

b End_Case ; “break out”

Case_B:

... ; any instruction sequence

b End_Case ; “break out”

Case_C:

... ; any instruction sequence

End_Case:



Hardware/Software Interface

Table based branching

tbb<c><q> [<Rn>, <Rm>] ; for tables of offset bytes (8bit)
tbh<c><q> [<Rn>, <Rm>, **lsl** #1] ; for tables of offset halfwords (16bit)

Common usage for halfword (16bit) tables

tbh [PC, Ri, **lsl** #1] ; PC used as base of branch table, Ri is index

Branch_Table:

.hword (Case_A - Branch_Table)/2 ; Case_A 16 bit offset
.hword (Case_B - Branch_Table)/2 ; Case_B 16 bit offset
.hword (Case_C - Branch_Table)/2 ; Case_C 16 bit offset

Case_A:

... ; any instruction sequence
b End_Case ; “break out”

Case_B:

... ; any instruction sequence
b End_Case ; “break out”

Case_C:

... ; any instruction sequence

End_Case:



Hardware/Software Interface

Basic instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	<code>add</code> , <code>adc</code> , <code>qadd</code> , <code>sub</code> , <code>sbc</code> , <code>qsub</code> , <code>rsb</code> , <code>mul</code> , <code>mla</code> , <code>mls</code> , <code>udiv</code> , <code>sdiv</code> , <code>umull</code> , <code>umlal</code> , <code>smull</code> , <code>smlal</code> , <code>and</code> , <code>bic</code> , <code>orr</code> , <code>orn</code> , <code>eor</code> , <code>cmp</code> , <code>cmn</code> , <code>tst</code> , <code>teq</code>
Move and shift registers		<code>mov</code> , <code>lsr</code> , <code>asr</code> , <code>lsl</code> , <code>ror</code> , <code>rrx</code>
Branching	Uses CPU flags	<code>b</code> , <code>bl</code> , <code>bx</code> , <code>blx</code> , <code>tbb</code> , <code>tbh</code>
Load & Store	Effects memory	<code>ldr</code> , <code>str</code> , <code>ldmdb</code> , <code>ldmia</code> , <code>stmia</code> , <code>stmdb</code>



Hardware/Software Interface

Basic instruction sets

Over 50 billion CPUs on this planet are running ARM instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	<code>add</code> , <code>adc</code> , <code>qadd</code> , <code>sub</code> , <code>sbc</code> , <code>qsub</code> , <code>rsb</code> , <code>mul</code> , <code>mla</code> , <code>mls</code> , <code>udiv</code> , <code>sdiv</code> , <code>umull</code> , <code>umlal</code> , <code>smull</code> , <code>smlal</code> , <code>and</code> , <code>bic</code> , <code>orr</code> , <code>orn</code> , <code>eor</code> , <code>cmp</code> , <code>cmn</code> , <code>tst</code> , <code>teq</code>
Move and shift registers		<code>mov</code> , <code>lsr</code> , <code>asr</code> , <code>lsl</code> , <code>ror</code> , <code>rrx</code>
Branching	Uses CPU flags	<code>b</code> , <code>bl</code> , <code>bx</code> , <code>blx</code> , <code>tbb</code> , <code>tbh</code>
Load & Store	Effects memory	<code>ldr</code> , <code>str</code> , <code>ldmdb</code> , <code>ldmia</code> , <code>stmia</code> , <code>stmdb</code>

Instruction sets in the field:

RISC: Power, ARM, MIPS, Alpha, SPARK, AVR, PIC, ...

CISC: x86, Z80, 6502, 68000, ...



Hardware/Software Interface

Basic instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	<code>add</code> , <code>adc</code> , <code>qadd</code> , <code>sub</code> , <code>sbc</code> , <code>qsub</code> , <code>rsb</code> , <code>mul</code> , <code>mla</code> , <code>mls</code> , <code>udiv</code> , <code>sdiv</code> , <code>umull</code> , <code>umlal</code> , <code>smull</code> , <code>smlal</code> , <code>and</code> , <code>bic</code> , <code>orr</code> , <code>orn</code> , <code>eor</code> , <code>cmp</code> , <code>cmn</code> , <code>tst</code> , <code>teq</code>
Move and shift registers		<code>mov</code> , <code>lsr</code> , <code>asr</code> , <code>lsl</code> , <code>ror</code> , <code>rrx</code>
Branching	Uses CPU flags	<code>b</code> , <code>bl</code> , <code>bx</code> , <code>blx</code> , <code>tbb</code> , <code>tbh</code>
Load & Store	Effects memory	<code>ldr</code> , <code>str</code> , <code>ldmdb</code> , <code>ldmia</code> , <code>stmia</code> , <code>stmdb</code>

What's missing?

- ✎ Changing CPU privileges and handling interrupts.
- ✎ Synchronizing instructions

Coming in later chapters
about concurrency and
operating systems.



Hardware/Software Interface

Summary

Hardware/Software Interface

- **Instruction formats**
 - Register sets
 - Instruction encoding
- **Arithmetic / Logic instructions inside the CPU**
 - Summation, Subtraction, Multiplication, Division
 - Logic and shift operations
- **Load / Store and addressing modes**
 - Direct, relative, indexed, and auto-index-increment addressing forms
- **Branching**
 - Conditional branching and unconditional jumps.